СЛАУ. Гаусс-Жордан. Фролов А.А., Курылев Г.А., 2к ИВТ-2

Ссылка на скринкаст: https://disk.yandex.ru/i/LPfFgs94N2koXA

Исходные данные

Математическая модель:

$$\tilde{a}_{ki} = \frac{a_{ki}}{a_{ii}}$$
$$a_{kj} = a_{kj} - \tilde{a}_{ki} * a_{ij}$$

$$i = 1 \div (n-1); k = 1 \div n; j = i \div (n+1); k \neq i$$

$$x_n = \frac{\tilde{a}_{i(n+1)}}{\tilde{a}_{ii}}$$
$$i = 1 \div n$$

Реализация

Код:

```python
# A = [[5, 7, 6, 5], [7, 10, 8, 7], [6, 8, 10, 9], [5, 7, 9, 10]]
# B = [23, 32, 33, 31]

A = [[4, 2, -1, 3], [3, 6, 2, -1], [2, -1, 5, 4], [1, 3, 2, 7]]
B = [6, 5, 24, 12]


def printMatrix(A):
    print("---------Матрица A---------\n")

    for i in range(len(A)):
        for j in range(len(A[i])):
            if j == len(A):
                print("B: ", round(A[i][j], 1), end=" ")
            else:
                print(round(float(A[i][j]), 1), end=" | ")
        print("\n")

    print("---------------------------")


def GaussJordan(A=None, B=None):
    if A is None or B is None:
        raise ValueError("A and B must be matrix")

```

```
    n = len(A)

    for i in range(len(A)):
        A[i].append(B[i])

    for i in range(0, n):
        for k in range(0, n):
            if k ≠ i:
                a_ki = A[k][i] / A[i][i]
                for j in range(i, n + 1):
                    A[k][j] = A[k][j] - a_ki * A[i][j]

    printMatrix(A)

    X = [0] * n

    for i in range(0, n):
        X[i] = A[i][n] / A[i][i]

    return X


X = GaussJordan(A, B)
for x in X:
    print(round(x, ndigits=1), end="\t")](<def Gauss_row_pivot(A=None, B=None):
    if A is None or B is None:
        raise ValueError("A and B must be matrix")

    n = len(A)

    # 1. Формируем расширенную матрицу [A | B]
    for i in range(n):
        A[i].append(B[i])

    # 2. Массив для отслеживания перестановки столбцов (перенумерация переменных)
    col_index = list(range(n))

    # 3. Прямой ход метода Гаусса
    for i in range(n - 1):

        # 3.1 Поиск максимального по модулю элемента в текущей строке
        max_col = i
        for j in range(i + 1, n):
            if abs(A[i][j]) %3E abs(A[i][max_col]):
                max_col = j

```

```python
        # 3.2 Перестановка столбцов (если найден элемент больше диагонального)
        if max_col != i:
            for k in range(n):
                A[k][i], A[k][max_col] = A[k][max_col], A[k][i]

            # фиксируем изменение порядка переменных
            col_index[i], col_index[max_col] = col_index[max_col], col_index[i]

        # 3.3 Обнуление элементов под главным элементом
        for k in range(i + 1, n):
            factor = A[k][i] / A[i][i]  # коэффициент исключения
            for j in range(i, n + 1):
                A[k][j] -= factor * A[i][j]

    # 4. Обратный ход (нахождение неизвестных)
    X = [0] * n
    for i in range(n - 1, -1, -1):
        s = A[i][n]
        for j in range(i + 1, n):
            s -= A[i][j] * X[j]
        X[i] = s / A[i][i]

    # 5. Возвращаем переменные в исходном порядке
    result = [0] * n
    for i in range(n):
        result[col_index[i]] = X[i]

    return result>)
```

Результат работы:

```
2.0     -1.0     3.0     1.0
```

Вывод

Мы успешно реализовали метод Гаусса-Жордана на языке python.